

COMP90038 Algorithms and Complexity

Recursion

Junhao Gan

Week 3 Lecture 6

Semester 1, 2026

Acknowledgement

Some of the contents in these slides are inspired by the following materials:

Recursion by Yufei Tao:

https://www.cse.cuhk.edu.hk/~taoyf/course/2100/23-spring/lec/0d_rec

In this lecture, we will introduce a **powerful** algorithm design technique called *recursion*.

If used properly, recursion can significantly **simplify** the design of an algorithm, and provide a new, elegant, and **neat prospective** to tackle the problems.

Often times, it leads to a simpler running-time analysis as well.

To see this, we will use recursion to “re-discover”:

- the **binary search algorithm** for the Dictionary Search Problem, and
- the **selection sort algorithm** for the Sorting Problem.

The Basic Idea of Recursion Design

In general, the design of a recursion consists of two main steps:

- **The Base Case:**

Solve the problem when the problem size n is *small enough*, e.g., $n = 1$ or $n = 0$. In this case, the problem is usually *trivial*.

- **The Inductive Case:**

Solve the problem with $n > 1$ by solving the same problem with a *smaller* input size n' .

The Dictionary Search Problem

The **Dictionary Search Problem**:

- Input:
 - a set S of n integers sorted in *ascending order* and stored *consecutively* in an array of length n ,
 - a search target integer v ;
- Goal: decide if $v \in S$.

Re-discover the Binary Search Algorithm

The Base Case ($n = 1$ or $n = 0$):

This case is trivial:

- if $n = 0$, then simply return “NO” (to the Dictionary Search Problem);
- if $n = 1$, simply compare v to the *only* integer s in the array S in $O(1)$ time:
 - if $v = s$, then return “YES”;
 - otherwise, return “NO”.

Re-discover the Binary Search Algorithm

The Inductive Case ($n > 1$):

- Compare v to the integer s in the *middle of the array*. If $v = s$, then return “YES” and done;
- Otherwise:
 - if $v < s$, solve the problem in the part of the array *before* s ;
 - if $v > s$, solve the problem in the part of the array *after* s .

The description of the Binary Search algorithm is now completed.

Cost Analysis (Revisit)

Assuming n is a power of 2 and define $C(n)$ as the worst-case cost of the binary search with problem size n ; we have the following *recurrence*:

$$C(0) = 1$$

$$C(1) = 1$$

$$C(n) = C(n/2) + 1$$

To solve the recurrence, we use the *inductive case* $C(n/2) = C(n/4) + 1$ to expand, and keep going:

$$\begin{aligned} C(n) &= C(n/2) + 1 \\ &= [C(n/4) + 1] + 1 \\ &= [[C(n/8) + 1] + 1] + 1 \\ &\vdots \\ &= \underbrace{[[\dots [C(1) + 1] + 1] + \dots + 1] + 1}_{1 + \log_2 n \text{ times}} = O(\log n) \end{aligned}$$

Cost Analysis (Revisit)

Suppose n is **not** a power of 2; there must exist n' which is the *least* power-of-two number and $n' \geq n$. More specifically, by definition, we have:

$$n' = 2^{\lceil \log_2 n \rceil} \leq 2^{(\log_2 n) + 1} = 2 \cdot 2^{\log_2 n} = 2n.$$

Therefore, we have:

$$\begin{aligned} C(n) &\leq C(n') \\ &= O(\log n') \quad (\text{we proved this in the previous slide}) \\ &= c_1 \cdot \log_2 n' + c_2 \quad (c_1 \text{ and } c_2 \text{ are some constants}) \\ &\leq c_1 \cdot \log_2(2n) + c_2 \\ &= c_1 \cdot (1 + \log_2 n) + c_2 \\ &= c_1 \cdot \log_2 n + c_1 + c_2 \\ &= O(\log n) \end{aligned}$$

The Sorting Problem

The Sorting Problem:

Given a set S of n integers stored consecutively in an array A of length n , the goal is to sort all these integers in S in *ascending order* and store them in A .

Re-discover the Selection Sort Algorithm

Question: What would be the base case that you can think of?

The Base Case ($n = 1$ or $n = 0$):

Nothing to sort, return right away.

Re-discover the Selection Sort Algorithm

Question: What would be the inductive case that you can think of?

The Inductive Case ($n > 1$):

- Scan all the integers in the array to identify the *smallest* integer $s_{\min}$
- Swap $s_{\min}$ with the integer in the *first* position of the array.
- Sort the rest $n - 1$ integers in the array starting at the second position.

Re-discover the Selection Sort Algorithm

Define $C(n)$ as the worst-case cost of the Selection Sort Algorithm. We have the following the recurrence:

$$C(0) = 1$$

$$C(1) = 1$$

$$C(n) = O(n) + C(n - 1)$$

Solving the recurrence by the *expansion trick*, we have:

$$C(n) = O(n) + C(n - 1)$$

$$= O(n) + O(n - 1) + C(n - 2)$$

$$\vdots$$

$$= \sum_{i=1}^n O(i) + C(n - n) = O(n^2)$$

Computing Large Power Number

Large Power Calculation:

Given a positive integer a , and a large positive number n , compute the value of a^n .

Question: Now is your turn. Design a recursive algorithm for solving this problem.

Computing Large Power Number

The Base Case ($n = 1$ or $n = 0$):

- if $n = 0$, return 1;
- if $n = 1$, return a .

Inductive Case ($n > 1$)

- Compute $s = a^{n-1}$;
- Return $a \cdot s$;

Computing Large Power Number

Define $C(n)$ as the worst-case cost of the above algorithm. We have:

$$C(0) = 1$$

$$C(1) = 1$$

$$C(n) = C(n-1) + 1$$

Solving the recurrence, we have: $C(n) \in O(n)$.

Question: Can we do better?

A Faster Algorithm

The base case is the same as before.

Inductive Case ($n > 1$)

- If n is **odd**, compute $s = a^{(n-1)/2}$ and return $a \cdot s^2$;
- Otherwise, i.e., n is **even**, compute $s = a^{n/2}$ and return s^2 ;

Question: What would be the cost recurrence?

A Faster Algorithm

Define $C(n)$ as the worst-case cost of the above algorithm. We have:

$$C(0) = 1$$

$$C(1) = 1$$

$$C(n) \leq C(n/2) + 1$$

Solving the recursion by the expansion trick, we have: $C(n) \in O(\log n)$.

Topics Next Week

Graphs, trees, graph traversal and allied algorithms.